Requirements Engineering: A Comprehensive Study Guide

THE CRITICAL FIRST STEP IN SOFTWARE DEVELOPMENT

Introduction to Requirements Engineering

Requirements Engineering (RE) is the critical first step in the software development lifecycle. It forms the foundation upon which all subsequent development activities are built. Without a clear and accurate understanding of what the software needs to do, the project is destined for significant challenges, including budget overruns, missed deadlines, and ultimately, a product that fails to meet user needs. This chapter will delve into the fundamental concepts of RE, its core activities, and the crucial distinction between different types of requirements.

1. What is Requirements Engineering?

Requirements Engineering can be defined as the process of defining, documenting, and maintaining requirements for a software system. It is the essential bridge that connects the real-world needs and expectations of stakeholders with the technical realities of software implementation.

Definition with Real-World Example: When Uber was first built, RE involved understanding that users needed: to see driver location in real-time, estimate arrival time, and pay without cash. Missing any of these would have made the app useless. These are the core needs that the software must address.

Core Purpose: To ensure that the software developed accurately reflects and satisfies the needs and goals of its users and stakeholders. It is about understanding *what* the system should do, not *how* it should do it.

Fundamental RE Activities: The RE process typically encompasses several key activities:

- **Elicitation:** Gathering requirements from stakeholders, users, and other sources.
- **Analysis:** Understanding, structuring, and prioritizing the gathered requirements.
- **Specification:** Documenting the requirements in a clear, concise, and unambiguous manner.
- **Validation:** Confirming that the documented requirements accurately reflect stakeholder needs and that the system, if built to these requirements, will be fit for purpose.

2. Functional vs Non-Functional Requirements

Requirements are broadly categorized into two types: functional and non-functional. Understanding this distinction is crucial for comprehensive system design.

Feature	Functional Requirements (FR)	Non-Functional Requirements (NFR)
Definition	Describe *what* the system should do (its behavior or functions).	Describe *how* the system should perform its functions (qualities).
Focus	Functionality and behaviors .	Performance, security, usability, reliability, maintainability, etc.
Characteristics	Often expressed as "The system shall..." followed by an action.	Often apply to the system as a whole rather than individual functions.
Examples	1. The system shall allow doctors to search the drug formulary. 2. The system shall generate a monthly report of patient visits.	1. The system shall respond to queries within 2 seconds. 2. Patient data must be encrypted to AES-256 standards.

Real-World Example: For Netflix:

- **FR:** 'Users shall be able to create up to 5 profiles per account.'
- **NFR:** 'Video streaming shall buffer in under 3 seconds on a 10 Mbps connection.'

3. Requirements Elicitation Techniques

Gathering requirements effectively requires employing various techniques to uncover the diverse needs of stakeholders. Each technique has its strengths and is best suited for different contexts.

- **Interviews:** Directly engaging with stakeholders to ask questions and gather information. This allows for in-depth understanding and clarification.
  - *Real-world example:* When building Apple Pay, developers interviewed cashiers and customers to understand payment friction points like fumbling for cards or slow chip readers.
- **Scenarios/User Stories:** Describing how a user will interact with the system to achieve a goal. User stories are a popular format in Agile development.
  - *Real-world example:* Airbnb user story: 'As a traveler, I want to filter by WiFi speed so I can

work remotely during my stay.'
- **Prototyping:** Creating a preliminary model or mock-up of the system to get user feedback early in the development process.
  - *Real-world example:* Instagram tested Stories as a prototype with select users before full launch, discovering users wanted the 24-hour disappearing feature.
- **Ethnography:** Observing users in their natural environment to understand their workflows, challenges, and unstated needs.
  - *Real-world example:* Amazon observed warehouse workers and discovered they needed hands-free scanning, leading to wearable barcode scanners instead of handheld devices.

4. Why Clear, Testable Requirements Matter

The significance of clear, unambiguous, and testable requirements cannot be overstated. Errors introduced early in the lifecycle are exponentially more costly to fix later.

The High Cost of Failure: It is widely recognized that a significant percentage of software defects originate in the requirements phase. Estimates suggest that **44-80% of errors** can be traced back to issues at this stage. Addressing these errors after development has progressed or the system has been deployed incurs substantially higher costs in terms of time, resources, and potential project failure.

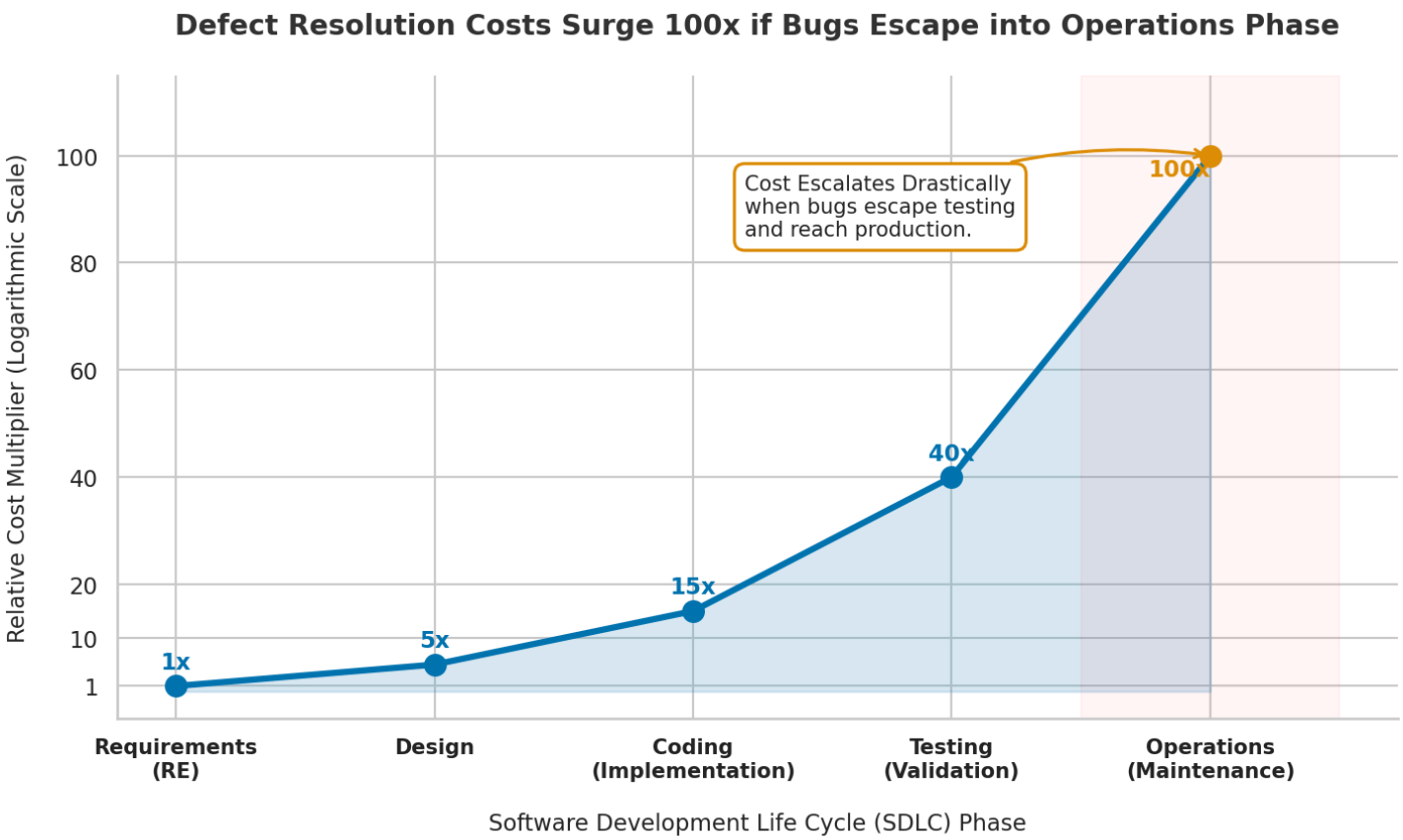
Real-World Example: Healthcare.gov's 2013 launch disaster cost an estimated $1.7 billion. A significant factor contributing to this was unclear requirements – specifically, a lack of definition for how many simultaneous users the system needed to handle, leading to catastrophic performance issues.

The Exponential Cost Curve of Bug Fixing

The graph below illustrates the dramatic increase in the cost to fix a software defect as it progresses through the Software Development Lifecycle (SDLC).

Graph Visualization:

- **Y-axis:** Relative Cost to Fix Bug
- **X-axis:** SDLC Phases (Requirements, Design, Code, Test, Operations)



Source: Grounded in Ian Sommerville, Software Engineering (Chapter 4) / Barry Boehm's Software Economics

The visual representation shows a steeply rising curve, starting with a minimal cost during the Requirements phase and escalating exponentially through Design, Code, and Testing, reaching its peak during the Operations (post-deployment) phase.

Real-world Example for Graph: A typo in requirements (fixing a field name) might cost as little as 1 hour of effort. The same error discovered after deployment requires extensive efforts such as database migration, code modifications across multiple modules, re-testing, and user retraining – potentially costing 100 times more than fixing it in the requirements phase.

5. Evaluating and Writing Good Requirements

To ensure project success, requirements must adhere to certain quality attributes. Writing good requirements is an art and a science that involves clarity, precision, and verifiability.

Key Quality Metrics:

- **Completeness:** All necessary information should be present. No missing functionality or constraints.
- **Unambiguity:** Each requirement should have only one possible interpretation.

Evaluation Table: Bad Requirement vs. Good Requirement

Requirement Type	Bad Requirement	Why it's Bad	Good Requirement

Search Speed	The system should be fast.	Vague, not measurable.	The system shall return search results for any valid query within 2 seconds.
Ease of Use	The user interface should be easy to use.	Subjective, not testable.	New users shall be able to complete a standard purchase transaction within 5 minutes without assistance.
Error Handling	The system should handle errors gracefully.	Ambiguous, lacks specifics.	If a user enters an invalid email format, the system shall display an error message: 'Invalid email format. Please correct.'
Security	Sensitive data must be protected.	Lacks specific protection mechanisms.	All user passwords shall be stored using bcrypt hashing with a salt.

Real-world Example: Tesla's Autopilot requirements specify exact conditions: 'Lane-keeping shall maintain vehicle within lane markings at speeds between 0-90 mph on highways with clear lane markings.' This is clear, testable, and defines the operating environment, unlike a vague requirement like 'keep car in lane.'

6. Key Takeaways for Review - Requirements Engineering Essentials

Requirements Engineering is foundational to successful software development. Prioritizing and executing RE activities effectively mitigates risks and ensures the final product aligns with stakeholder expectations.

Memorable Example for Review: Think of RE like ordering at a restaurant:

- **Vague Requirement:** Saying 'I want something good' leaves the chef guessing and is unlikely to satisfy you.
- **Clear, Testable Requirement:** Saying 'I want a medium-rare ribeye steak with garlic mashed potatoes, no butter' provides specific, verifiable details that ensure you get exactly what you

expect. Similarly, RE demands precise and testable specifications for software.